CLOUD COMPUTING

Vidya Vikas Education Society's

VIKAS COLLEGE OF ARTS, SCIENCE & COMMERCE

Affiliated to University of Mumbai

RE-ACCREDITED BY NAAC

ISO 9001 : 2008 CERTIFIED

High School Marg, Kannamwar Nagar No 2, Vikhroli (E), Mumbai – 400083

This is to certify that,

Student of T.Y.B.Sc. (**Computer Science**) (Sem-VI) with college Roll No. _____ and University of Mumbai Seat No. _____ has satisfactorily completed the practical work for the Subject **Cloud Computing** in the program of Computer Science from the **UNIVERSITY OF MUMBAI** for the academic year 2025-2026.

_____ _____

Guided By **Head Of Department**

Place – Mumbai

Date -

INDEX

S.NO	Practical Name	Date	Sign
1	Define a simple services like Converting Rs into Dollar and Call it from different platform like JAVA and .NET		
2	Create a Simple SOAP service.		
3	Create a Simple REST Service.		
4	Develop application to consume Google's search / Google's Map RESTful Web service.		
5	Installation and Configuration of virtualization using KVM.		
6	Develop application to download image/video from server or upload image/video to server using MTOM techniques.		
7	Implement FOSS-Cloud Functionality VSI (Virtual Server Infrastructure) Infrastructure as a Service (IaaS), Storage.		
8	Implement a FOSS –cloud functionality –VSI Platform as a service.		

PRACTICAL 1

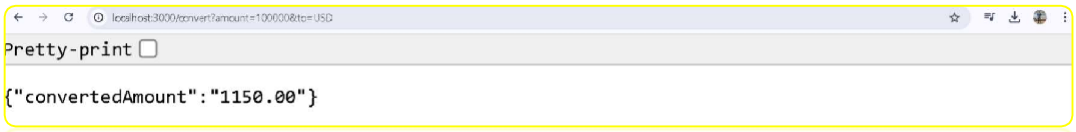
- **Define a simple services like Converting Rs into Dollar and Call it from different platform like JAVA and .NET**

```
const express = require('express');
const axios = require('axios');
const app = express();

const exchangeRateApi = 'https://api.exchangerate-api.com/v4/latest/INR';

app.get('/convert', async (req, res) => {
  const { amount, to } = req.query;

  try {
    const response = await axios.get(exchangeRateApi);
    const exchangeRate = response.data.rates[to];

    if (!exchangeRate) {
      return res.status(400).json({ error: 'Invalid currency code' });
    }

    const convertedAmount = (amount * exchangeRate).toFixed(2);
    res.json({ convertedAmount });
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'An error occurred while converting currency' });
  }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});

// Keep the process alive
process.on('SIGINT', () => {
  console.log('Server shutting down...');
  process.exit(0);
});
```

Go to desktop>cmd> type node filename.js (node app.js) Copy port no (3000)

```
PS F:\SEM 6\Practicals\CCWS> node app.js
Server listening on port 3000
```

Go to browser type--- localhost:3000/convert?amount=100000&to=USD



PRACTICAL 2

❖ Create a Simple SOAP service.

```
const express = require("express");

var app = express();

app.get("/add/:num_1/:num_2", function(req, res) {

const num1 = req.params.num_1;

const num2 = req.params.num_2;

const result = parseInt(num1) + parseInt(num2);

res.json({result: result});

})

app.get("/sub/:num_1/:num_2", function(req, res) {

const num1 = req.params.num_1;

const num2 = req.params.num_2;

const result = parseInt(num1) - parseInt(num2);

res.json({result: result});

})

app.get("/mult/:num_1/:num_2", function(req, res) {

const num1 = req.params.num_1;

const num2 = req.params.num_2;

const result = parseInt(num1) * parseInt(num2);

res.json({result: result});

})

app.get("/div/:num_1/:num_2", function(req, res) {
```

```
const num1 = req.params.num_1;

const num2 = req.params.num_2;

const result = parseInt(num1) / parseInt(num2);

res.json({result: result});

})

app.listen(3300, function() {

   console.log("The port is 3300");

   });
```

Run in vs code copy the local host port number

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hardik Kanekar> node "c:\Users\Hardik Kanekar\Downloads\tempCodeRunnerFile.js"
The port is 3300
```

Open google chrome type—

http://localhost:3300/add/2/3

http://localhost:3300/sub/2/3

http://localhost:3300/mult/2/3

http://localhost:3300/div/2/3

```
localhost:3300/add/2/3
Pretty-print
{"result":5}
```

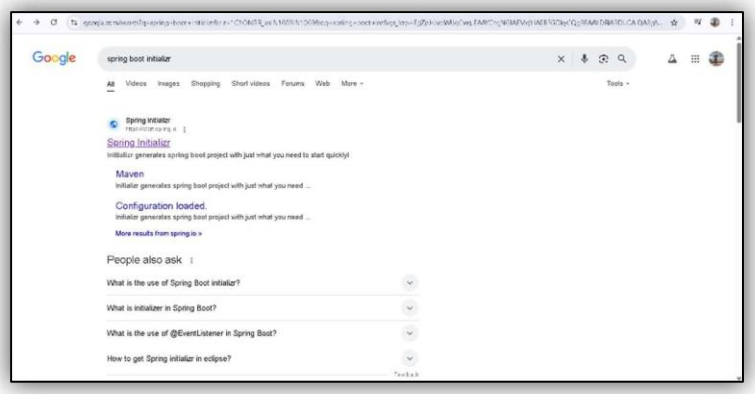
```
localhost:3300/sub/2/3
Pretty-print
{"result":-1}
```

```
localhost:3300/mult/2/3
Pretty-print
{"result":6}
```

```
localhost:3300/div/2/3
Pretty-print
{"result":0.6666666666666666}
```

PRACTICAL 3

❖ Create a Simple REST Service.

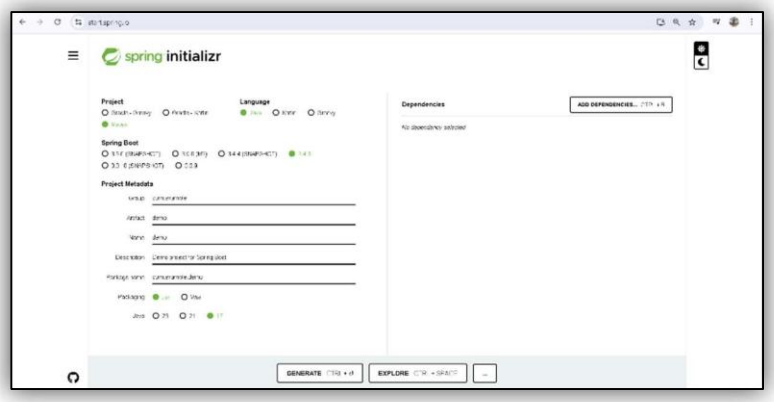


SELECT>PROJECT:MAVEN

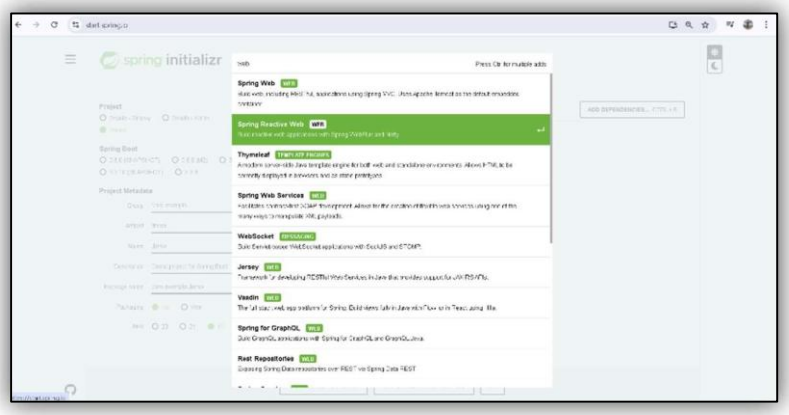
LANGUAGE:JAVA

SPRING BOOT :3.2.3

FILL UP PROJECT METADATA

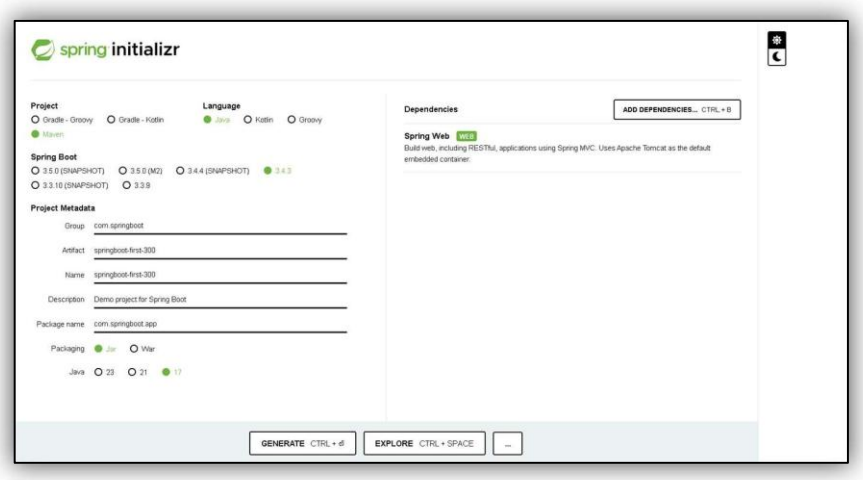


GO TO DEPENDENCIES >SEARCH WEB



mi

ADD DEPENDENCIES

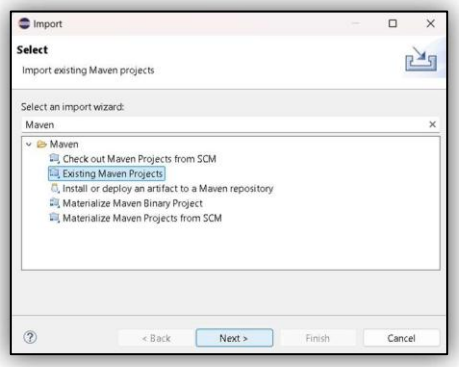


CLICK ON GENERATE BUTTON (BELOW) TO DOWNLOAD SPRING BOOT PROJECT AS ZIP FILE

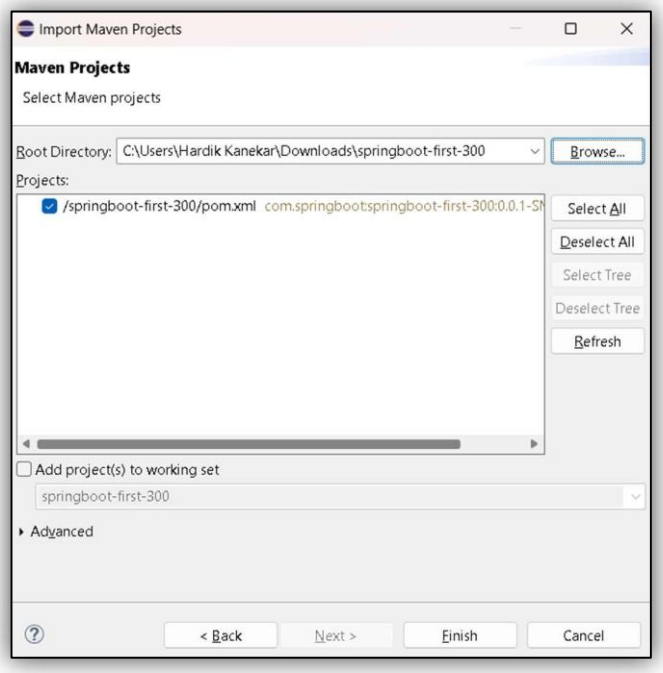
EXTRACT THE ZIP FILE

OPEN ECLIPSE>GO TO FILE>IMPORT

SELECT EXISTING MAVEN PROJECT

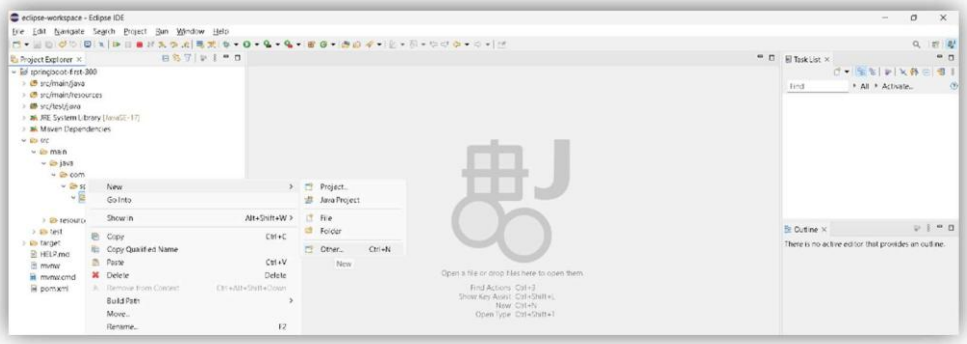


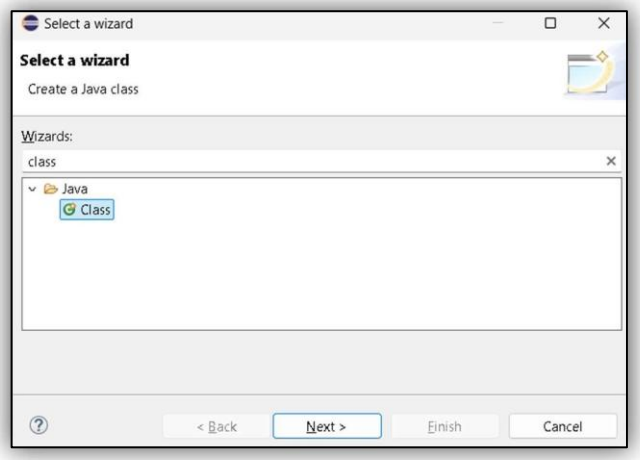
BROWSE SPRINGBOOT-FIRAT-APP >CLICK ON FINISH

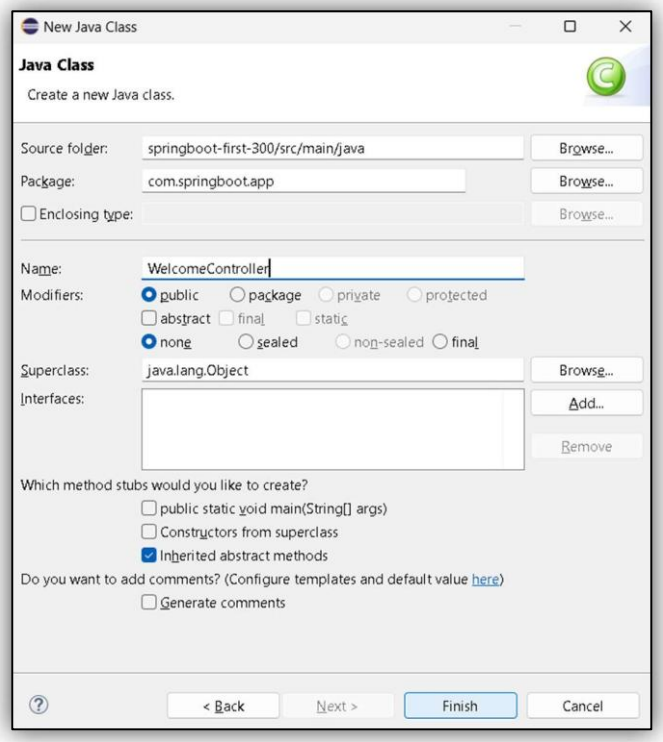


SELECT SPRINGBOOT-FIRST-APP>SRC>MAIN>JAVA>COM>SPRINGBOOT>APP

RIGHT CLICK ON APP>NEW>OTHERS>SELECT CLASS>NEXT



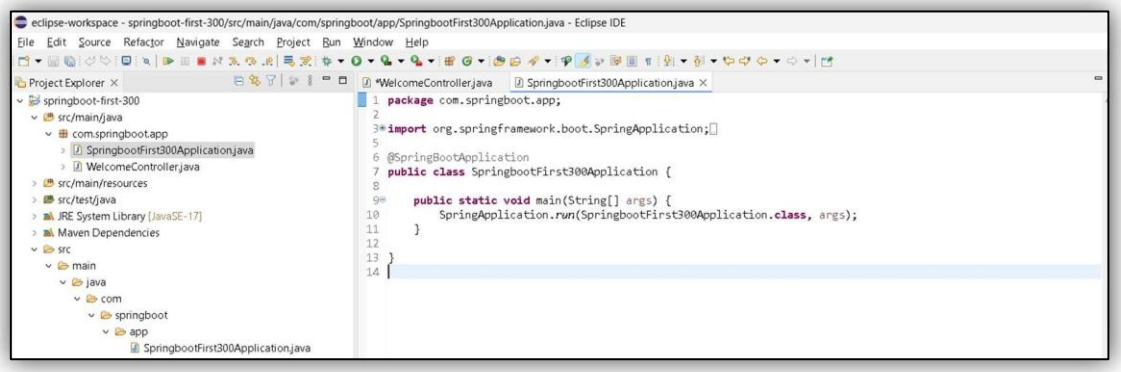




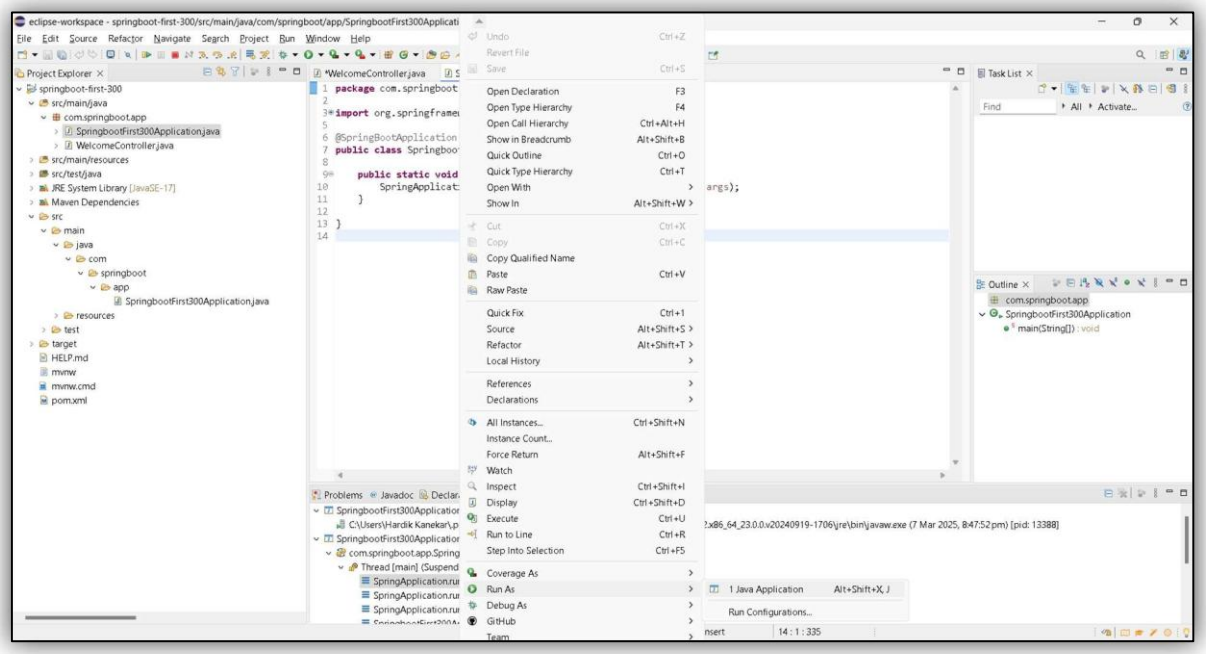
WRITE CODE IN WELCOMECONTROLLER.JAVA FILE

```
package com.springboot.springboot_10;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class WelcomeController {
    @GetMapping("/welcome")
    public String welcome() {
        return "welcome to spring booT";
    }
}
```

NOW GO TO springbootfirstappapplication.java



RUN THE PROGRAM (AS JAVA APPLICATION)



GO TO BROWSER TYPE>localhost:8080/welcome

Output :-

PRACTICAL 4

- **Develop application to consume Google's search / Google's Map RESTful Web service.**

```
import requests


def get_geolocation(api_key, search_string):
    base_url = "https://us1.locationiq.com/v1/search.php"  # Correct endpoint
    params = {
        'key': api_key,
        'q': search_string,
        'format': 'json',
    }

    response = requests.get(base_url, params=params)

    if response.status_code == 200:
        data = response.json()
        if data and isinstance(data, list):  # Ensure data is a valid list
            result = {
                'place_id': data[0].get('place_id', ''),
                'lat': data[0].get('lat', ''),
                'lon': data[0].get('lon', ''),
                'display_name': data[0].get('display_name', ''),
            }
            return result
        else:
            print("Error: Unexpected response format")
    else:
        print(f"Error: {response.status_code} - {response.text}")

    return None
```

```
# Corrected API Key (removed trailing space)
api_key = 'pk.71c93f03731ac10faf75d7e071df51c1'

# User input
search_string = input("Enter the location: ")

# Fetch and display the geolocation
result = get_geolocation(api_key, search_string)

if result:
    print("\nOutput:")
    for key, value in result.items():
        print(f"{key}: {value}")
```

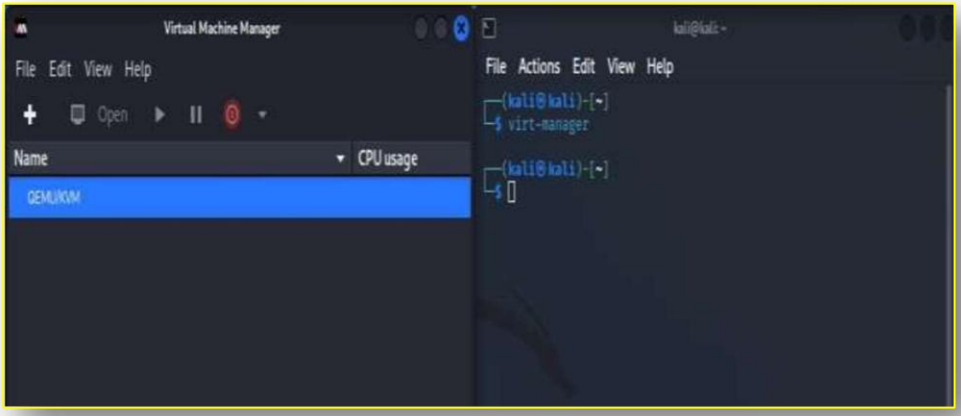
```
PS D:\Clound computing> python -u "d:\Clound computing\4.py"
Enter the location: Mumbai

Output:
place_id: 227883413
lat: 19.08157715
lon: 72.88662753964906
display_name: Mumbai, Maharashtra, India
PS D:\Clound computing>
```

PRACTICAL 5

❖ Installation and Configuration of virtualization using KVM.

1. sudo grep-c"svm\|vmx"/proc/cpuinfo
2. sudo apt install qemu-kvm libvirt-daemon-system virt-manager bird
3. sudo apt-get update
4. sudo apt-get install qemu-kvm libvirt-daemon-system virt-manager bridge-utils
5. sudo apt install qemu-kvm libvirt-clients libvirt-daemon-system bridge-utils
6. sudo systemctl start libvirtd
7. sudo usermod -aG kvm $USER
8. sudo systemctl is-active libvirtd
9. sudo usermod -aG libvirt $USER sudo usermod -aG kvm $USER
10. virt-manager
11. kvm-ok

Output :-



PRACTICAL 6

- **Develop application to download image/video from server or upload image/video to server using MTOM techniques.**

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const app = express();
const port = 3000;

// Serve a welcome page on the root route
app.get('/', (req, res) => {
  res.send('Welcome to the File Upload and Download API!');
});

// Define storage using multer.diskStorage
const storage = multer.diskStorage({
destination: (req, file, cb) => {
    // Set the destination folder where the file will be saved
    const uploadFolder = 'uploads';
    // Ensure the folder exists (asynchronously)
    fs.promises.mkdir(uploadFolder, { recursive: true })
      .then(() => cb(null, uploadFolder))
      .catch((err) => cb(err));
  },
  filename: (req, file, cb) => {
    // Sanitize the filename to avoid any path traversal issues
    const sanitizedFilename = path.basename(file.originalname);
    cb(null, sanitizedFilename);
  },
});
```

```
// Initialize multer with the defined storage configuration
const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // Limit file size to 5MB
  fileFilter: (req, file, cb) => {
    // Only allow specific file types (e.g., .jpg, .png)
    const allowedTypes = /jpeg|jpg|png/;
    const extname = allowedTypes.test(path.extname(file.originalname).toLowerCase());
    const mimeType = allowedTypes.test(file.mimetype);

    if (extname && mimeType) {
      return cb(null, true);
    } else {
      return cb(new Error('Only .jpg, .jpeg, .png files are allowed.'));
    }
  },
});

// Upload file route
app.post('/upload', upload.single('file'), (req, res) => {
  const file = req.file;
  // Check if the file is present
  if (!file) {
    return res.status(400).json({ success: false, message: 'No file uploaded.' });
  }
  // Send response after successful upload
  res.json({ success: true, message: 'File uploaded successfully.' });
});

// Download file route
app.get('/download/:filename', (req, res) => {
const filename = req.params.filename;
```

```
  const filePath = path.join(__dirname, 'uploads', filename);

  // Check if the file exists
  if (fs.existsSync(filePath)) {
    // Send the file as a response
    res.sendFile(filePath);
  } else {
    res.status(404).json({ success: false, message: 'File not found.' });
  }
});

// Start the server
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

RUN THE CODE: IT WILL START THE SERVER AT localhost:3000

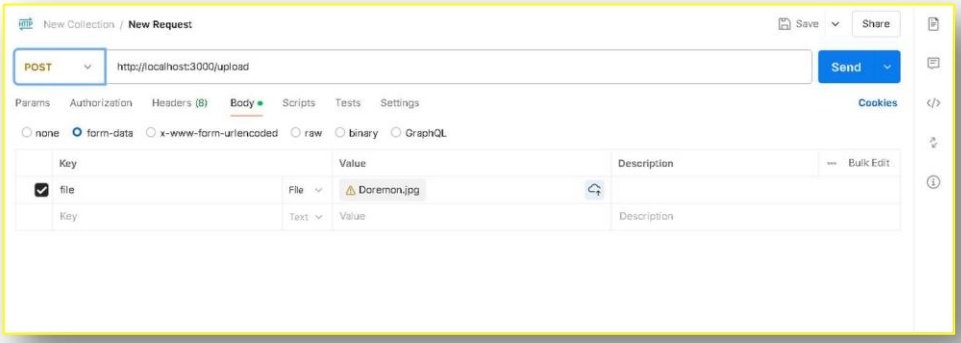
OPEN POSTMAN OPEN A NEW PAGE & CHOOSE POST METHOD

ENTER THE URL OF THE SERVER: http://localhost:3000/upload

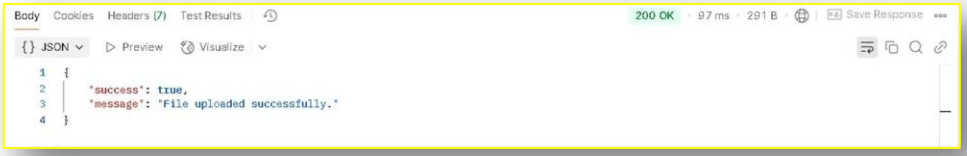
THEN CLICK ON BODY>FORM DATA>NAME THE KEY ITEM AND FILE TYPE: FILE

IN VALUE TAB ENTER THE FILE YOU WANT TO UPLOAD TO THE SERVER

CLICK ON SEND



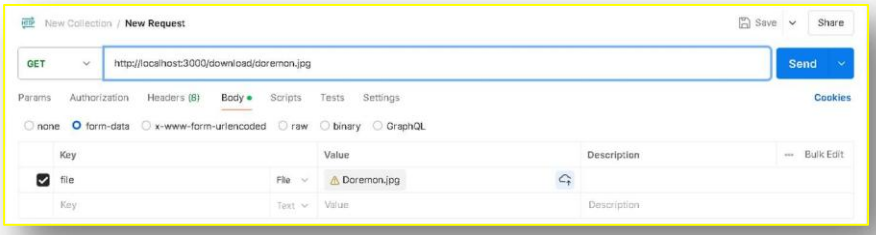
Upload Output :-



DOWNLOAD :-

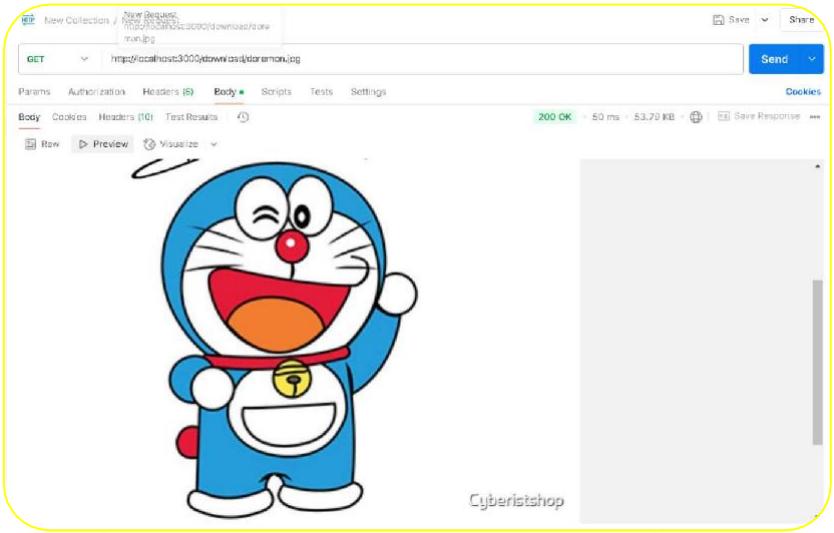
CHOOSE GET METHOD

ENTER THE URL OF THE SERVER: http://localhost:3000/download/doremon.jpg



CLICK ON SUBMIT

DOWNLOAD OUTPUT :-

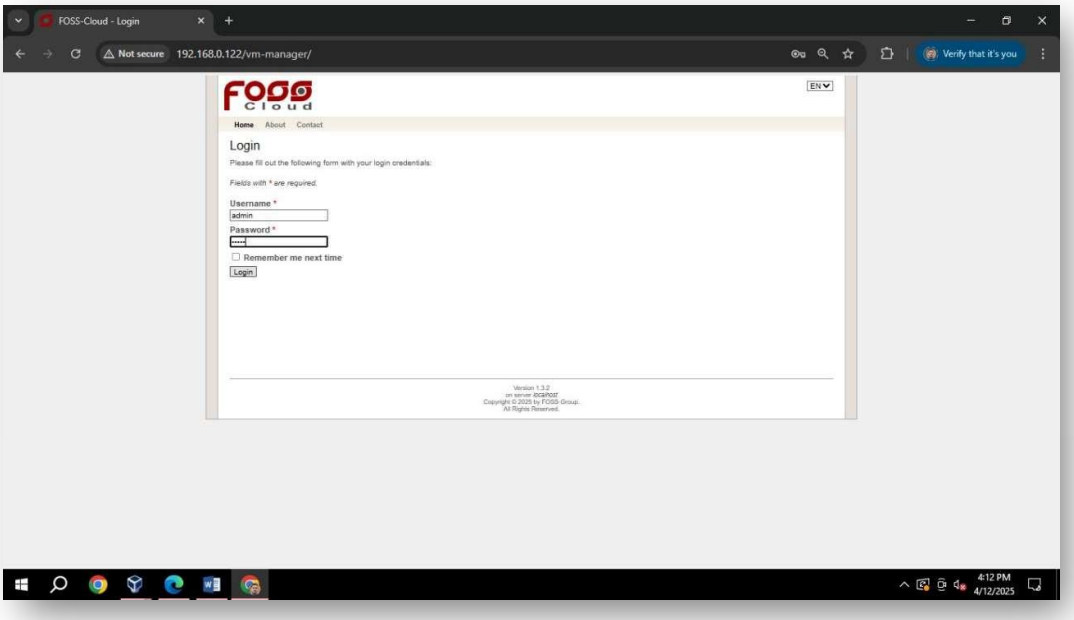


PRACTICAL 7

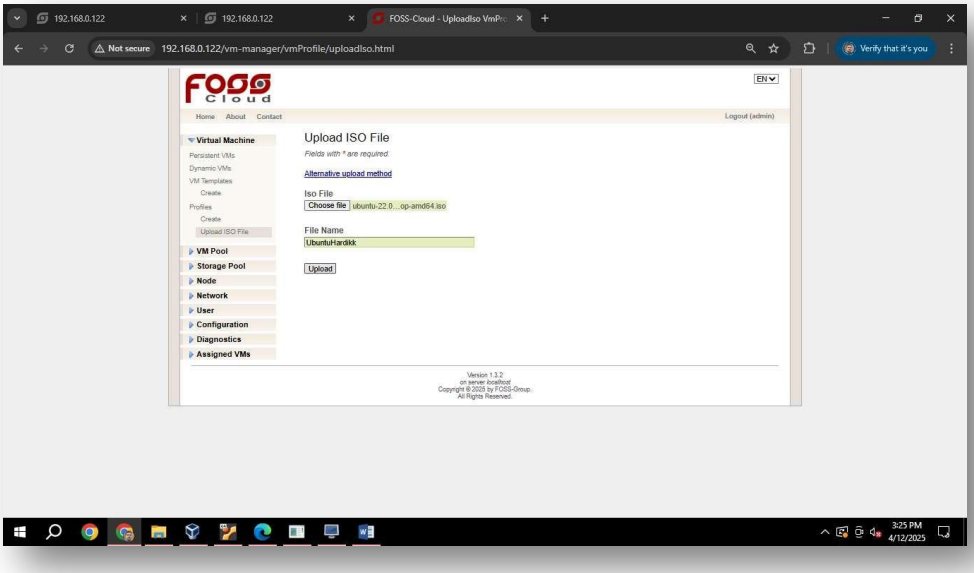
❖ Implement FOSS-Cloud Functionality VSI (Virtual Server Infrastructure) Infrastructure as a Service (IaaS), Storage

After installation, it will show you an ip address. Put it I'm your browser to access your administrator page the default user credentials are user: admin and password: admin. For root login - username- root and password - password.

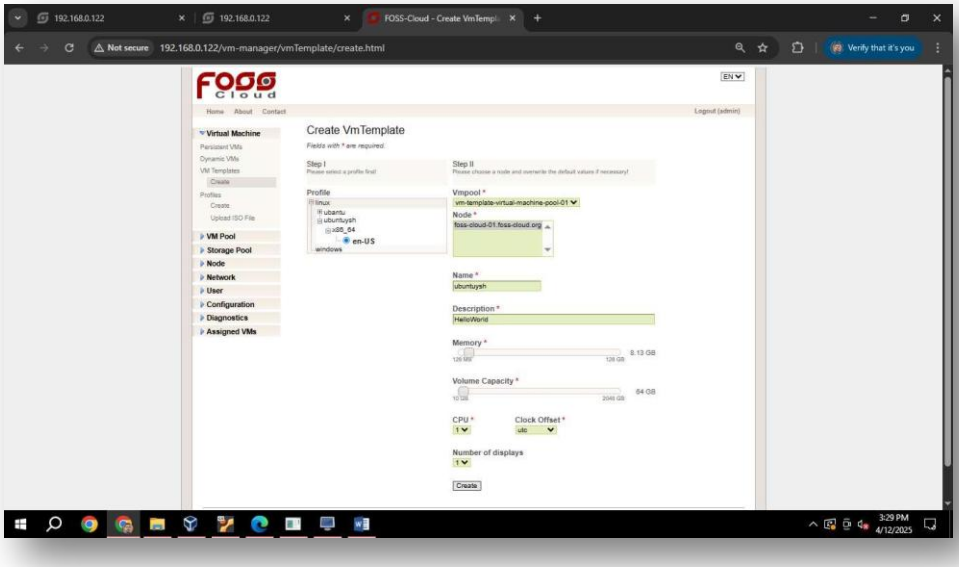
Go To Browser and Search 198.168.0.122



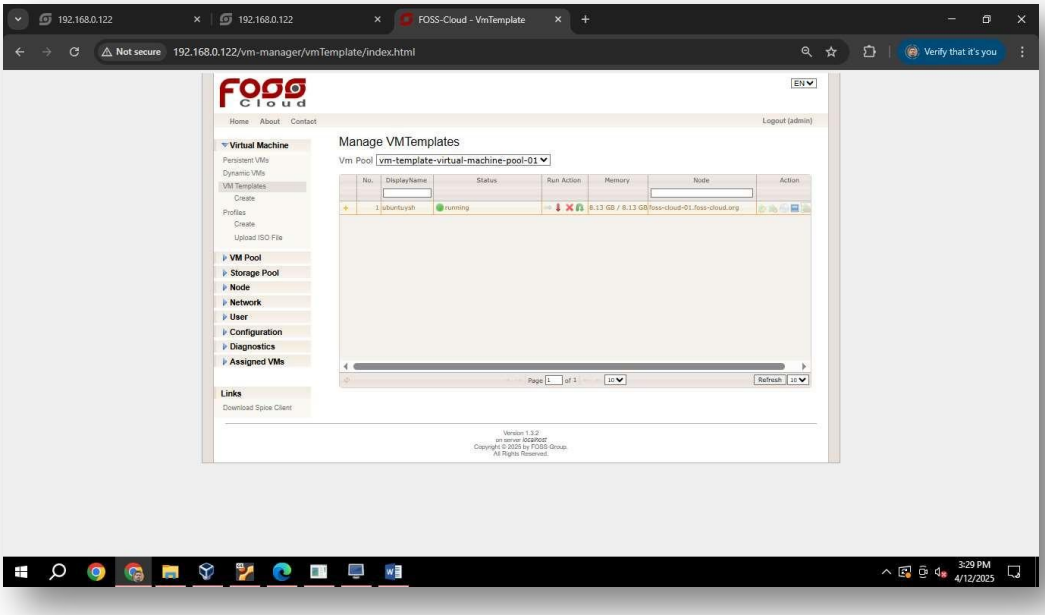
The first screen after login shows many options to install and deploy any virtual machine. To install a virtual machine click on virtual machine-> upload iso file option and upload the bootable iso file. Here, we are going to upload Linux elementary 0s is0.

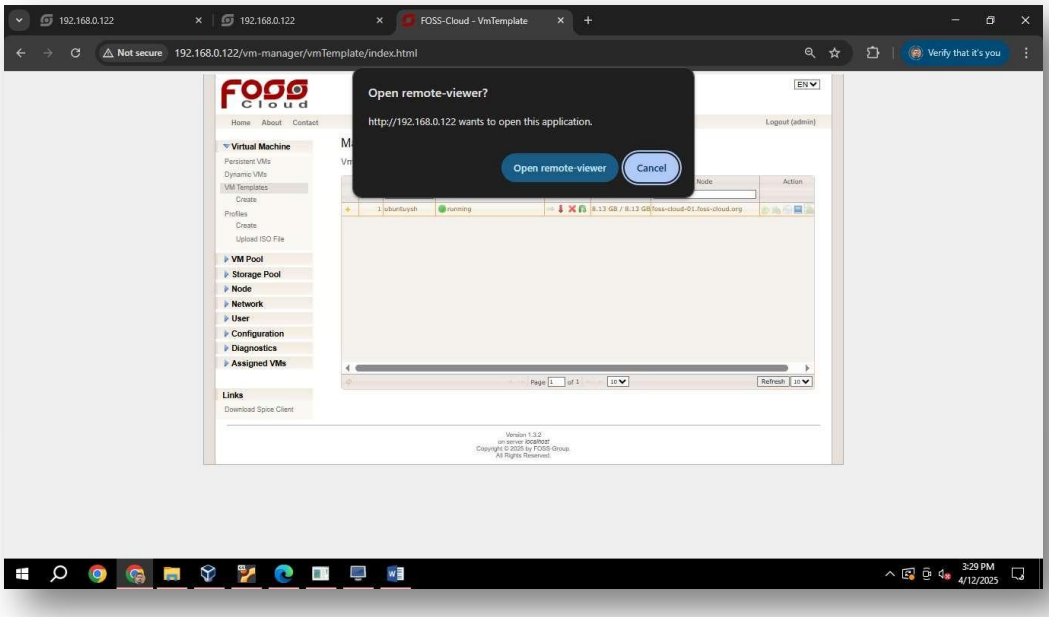


Once you uploaded the file, create vimtemplate. In this option you are basically configuring your virtual machine's storage location, cpu, memory, node etc. Here, you will find single nodes ana vw po01 1 respective options because everything was installed at the single server.

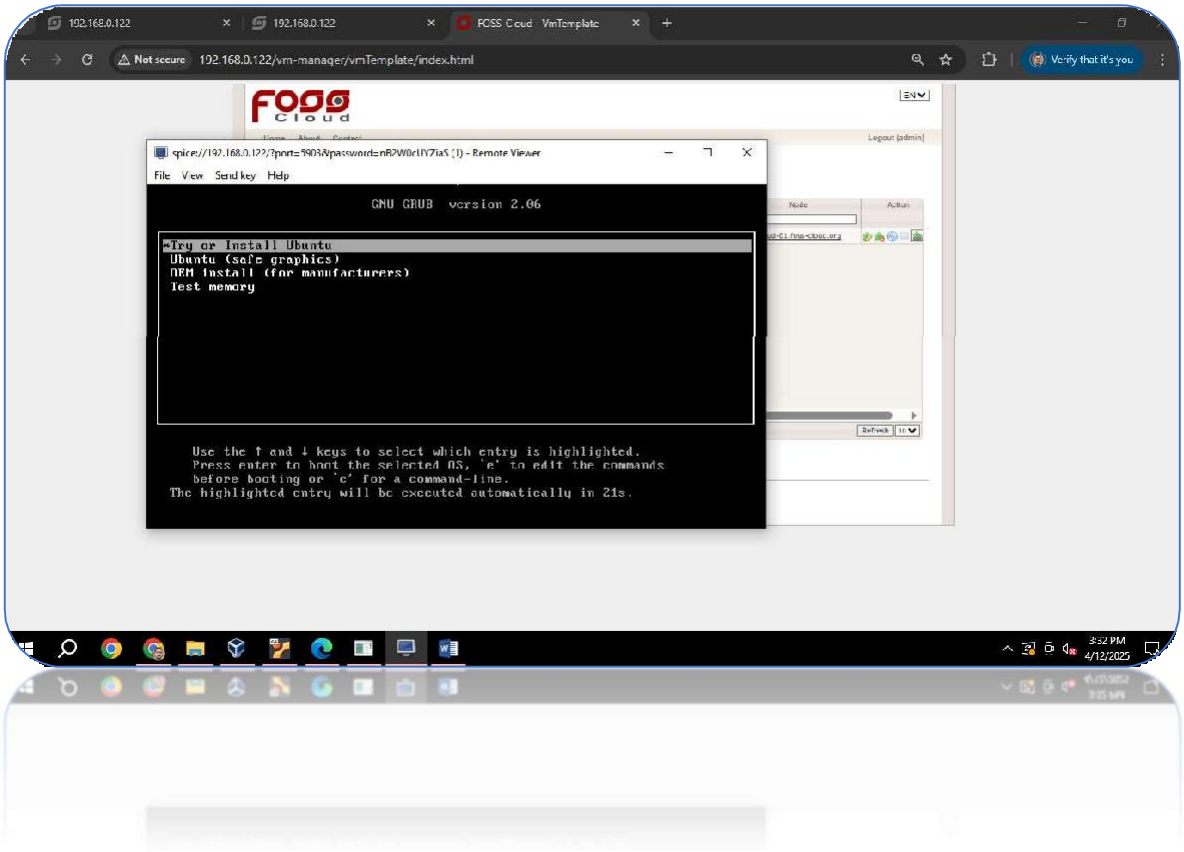


Now, click on vmtemplates and you will see a template which you have created in step 4. To start your machine go to run action tab and click on the green arrow. Under status tab, it shows the running text with the green circle which shows that your machine is running without any errors. To view your virtual machine click on a blue square box under action tab.





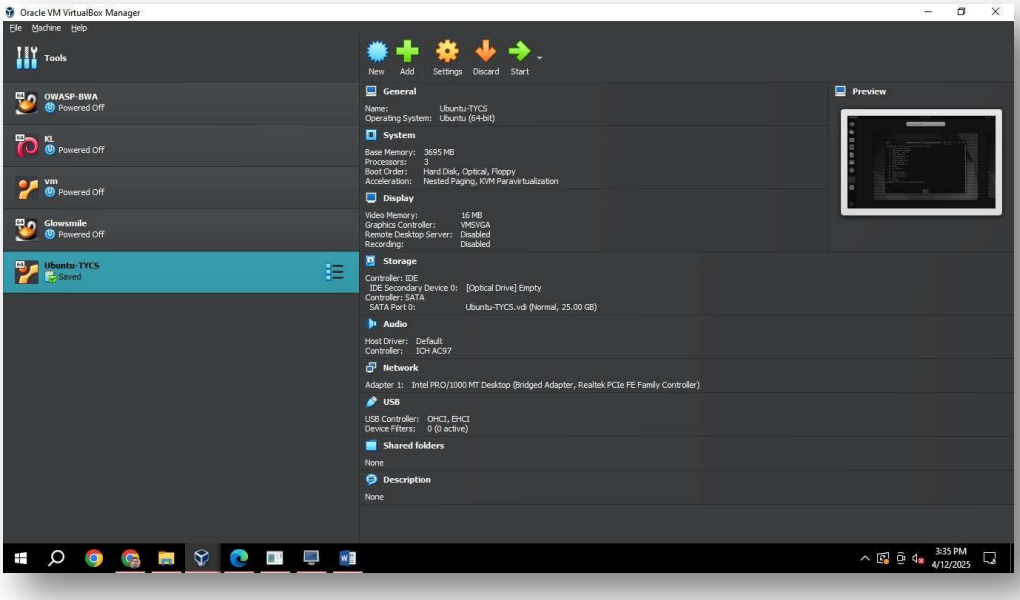
To view your virtual machine you have to download spice client tool. The download link can be found under the links option. After downloading the application, click on the blue square to view machine. When you click on it, the browser will pop-up for launching the application. If the application does not launch automatically, launch it manually by entering the link and password in the remote viewer tool which you will see in the pop-up message.

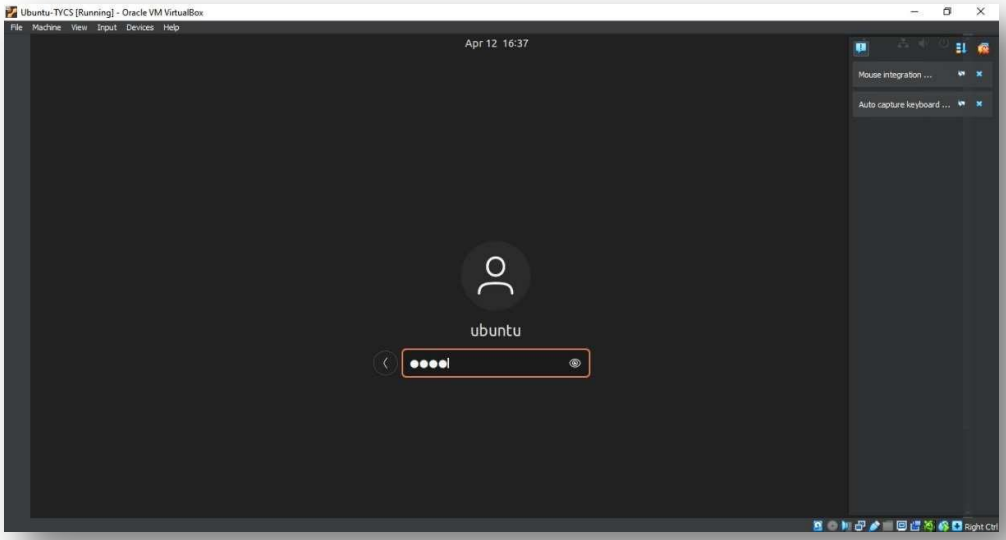


PRACTICAL 8

❖ Implement a FOSS –cloud functionality –VSI Platform as a service.

Step 1: Open Ubuntu

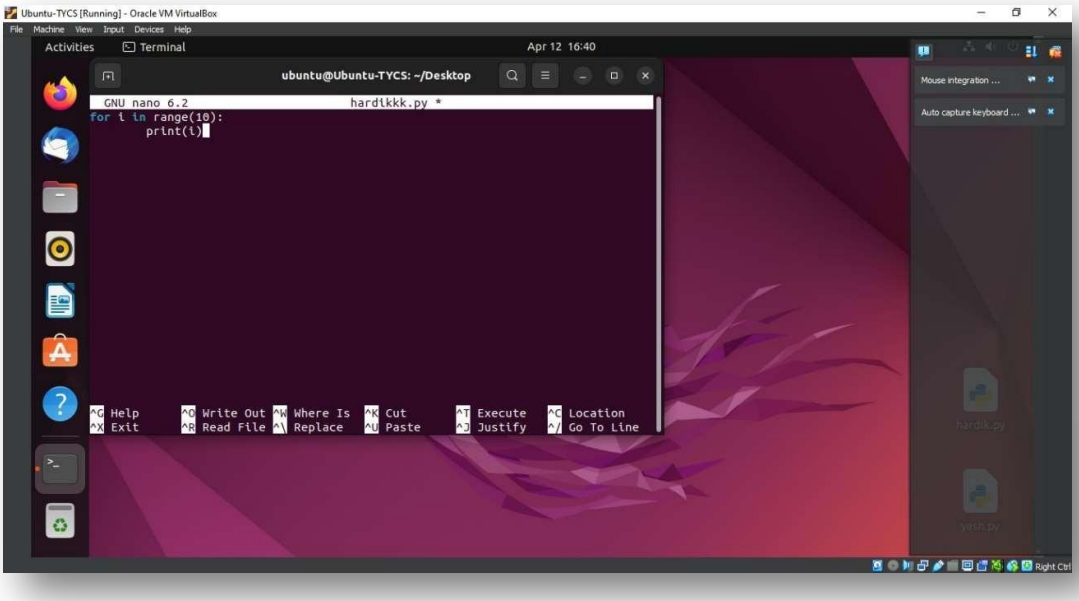
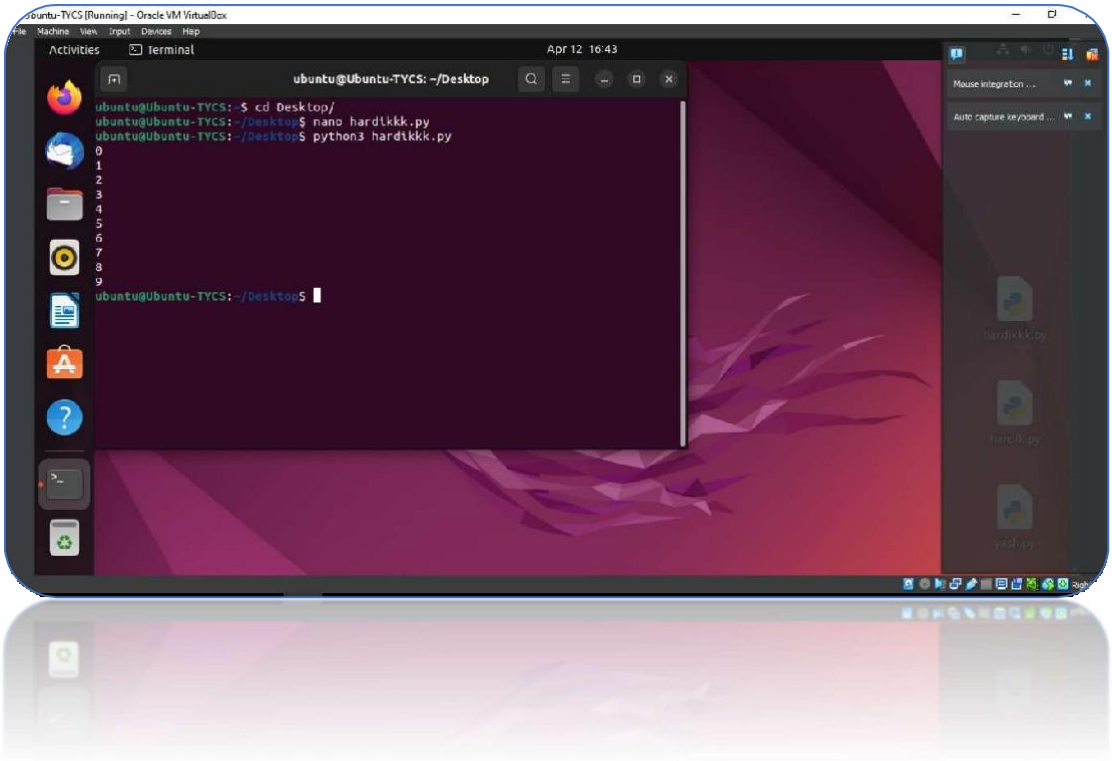




Step 2: Access the Ubuntu Terminal. Application -> Accessories -> Terminal

Step 3: Check for python.

Step 4: Create a Demo File for python, using Command gedit filename.py.



Step 5: Run Python File by using command python filename.py